MQ National Lottery (NL) Betting System Performance Test Plan

Prepared on	06 Oct 2025
Prepared by	Ronald Thian (Solutions Architect)
Vetted by	Dominic Tan (Project Manager)

Summary

This test plan outlines the performance testing strategy for the Betting System within the National Lottery (NL) project, specifically focusing on the placement and commitment of bets on the blockchain. This document will describe the objectives, setup, and strategy for evaluation of the system's baseline and high-rate order handling capabilities. The test environment used is an isolated environment with properties to mimic the production environment as closely as possible. Test strategy will detail how the betting function is triggered and then measured, using the appropriate monitoring tools. Several permutations of test cases will also be defined, from varying rates, to varying connections, in order to provide a comprehensive evaluation on the Betting System's performance under different loads and configurations.

This test is a follow up on the test conducted in May. A list of optimizations have been implemented in order to improve overall throughput:

1. Merger of pre-call APIs (e.g. `GET /partner/lottery/bet/betslip/seq`, `GET /partner/lottery/nextDraw`) onto the final `/placeBet` endpoint to reduce network bandwidth
    a. Removal of validation previously required to validate previous results
2. Indexing of database tables for more efficient querying, by adding queries with draw date filter

Objectives

To evaluate and provide accurate estimates of the performance of the Betting System in NL Project, hosted on the cloud.

Quantitative objectives are defined as follows:

Objective	Description	Key indicators
Transaction Volume	Evaluate the system's ability to handle 10,000 simultaneous transactions.	Txn volume in TPS
Response Time	Ensure each transaction is processed within a maximum of three (3) seconds.	Response time in seconds
Blockchain Integration	Verify the successful placement and commitment of bets on the blockchain can keep up with the expected transaction volume.	Committed txns volume in TPS
System Stability	Assess the overall stability and performance of the NL system under stress testing load.	Time to crash in seconds

Test Setup

Test component	Description
Load Generator	Tool to simulate concurrent users placing bets.
Monitoring Tools	Kubernetes Dashboard is used to monitor system performance and application pod health. JMeter will be used to perform the performance testing and record relevant round trip response time.
Test Environment	Dedicated environment mirroring production will be spun for accurate testing, including the application pods and database.
Blockchain Integration	The blockchain integration for this performance test has been omitted as it is managed by an external service provider (i.e. Kaleido) and the TPS can be accurately calculated based on the network node setup. Additionally, the infrastructure leverages a Kafka topic to stream requests into Kaleido and does not affect system performance.

General Test Parameters and Configuration

The following configurations are used for setting up of the test environment, in order to ensure that the test environment is as close to the production environment as possible:

Parameter	Production	Test
AWS RDS Size	db.large	db.large
Kubernetes HPA Minimum	2	2
Kubernetes HPA Maximum	4	4
Application Resource Quota	1-2 CPU Cores	2-4 CPU Cores

Test Strategy

Test Method

The Bet Type used for testing is as follows:

Game Type	2D, 3D, 4D, 6D, Lotto 6/42, Mega Lotto 6/45, Super Lotto 6/49, Grand Lotto 6/55, Ultra Lotto 6/58
Bet Amount	10 Pesos
Selected number	Random

Test Preparation:

1. Creation of User Account
2. Preparation of Lottery Parameters

Test sequence:

1. Place Bet
    a. Random Game Type
    b. Single Bet
    c. 10 Pesos
    d. Random selected number

Test Clients:

1. Test clients are running from a different AWS VPC, outside of where the test environment is set up.

2. A single test client is used, with thread count up to 1000
3. Round trip latency between the test client and the test environment is around 423 ms to 724 ms (tested with `curl` with message size of approx. 5 kB)

Example Scenarios:
Test Client = JMeter

Total web request server received = **500**,
Total number of user accounts = **1,**
Total bet order inserted into DB = **100**

1. Test Client login 1 user, initiate 1 session with the user token, resulting in 1 session token
2. Run place bet test script,
    a. Place Bet
3. Restart Betting System Application Pods

Total web request server received = **500**,
Total number of user accounts = **500,**
Total bet order inserted into DB = **500**

1. Test Client login 500 user, initiate 1 session with each user token, resulting in 500 session tokens
2. Run place bet test script,
    a. Place Bet
3. Restart Betting System Application Pods

Latency Calculation

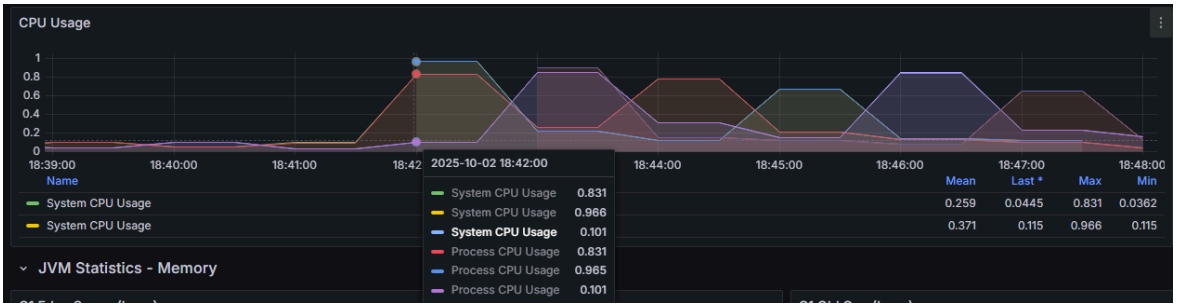
Latency is calculated using the `curl` command, extracting the following keys:

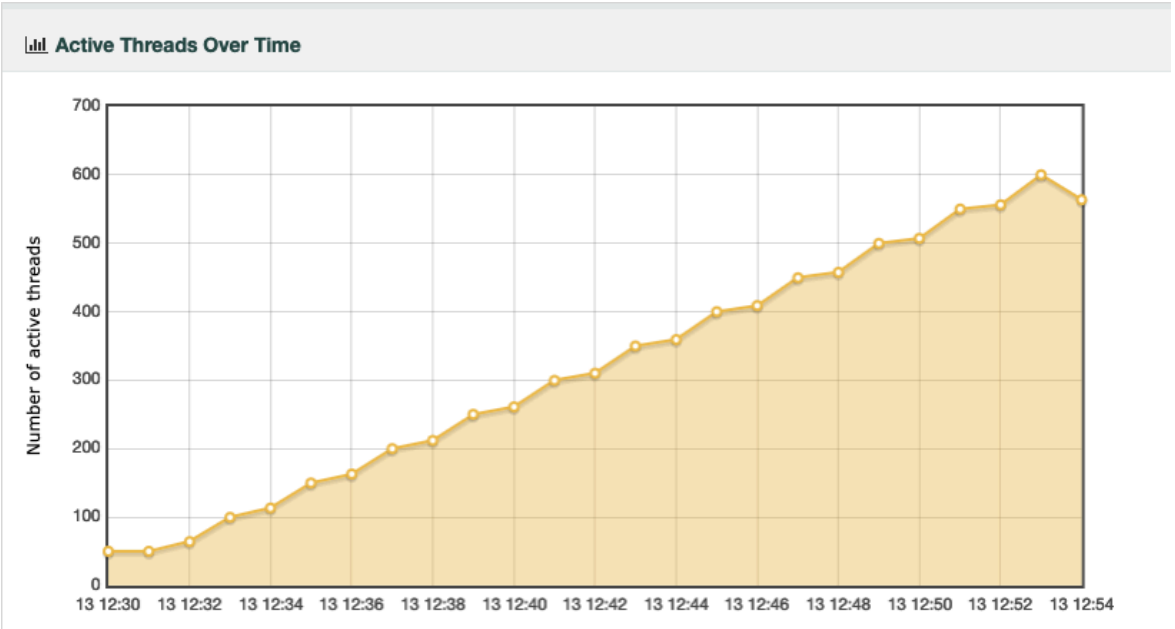
1. `time_namelookup` - Domain Name Server (DNS) Response
2. `time_connect` - Transmission Control Protocol (TCP)
3. `time_appconnect` - Transport Layer Security (TLS)
4. `time_starttransfer` - Time To First Byte (TTFB)
5. `time_total` - Total

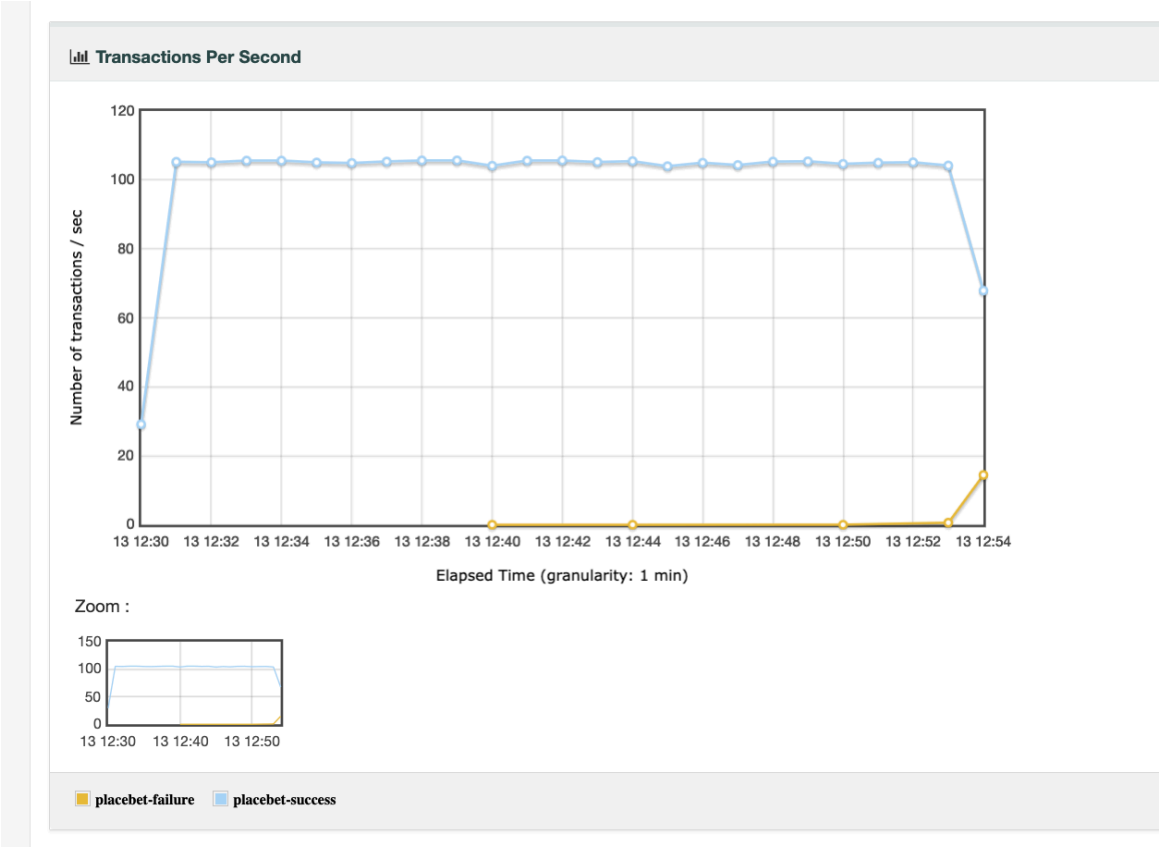
Basic Rate Test Cases

Test Case ID	Throughput (bets/sec)	Connection Amount
BR-V2-001*	-	600 (Target)
BR-001	100	50
BR-002	500	250

* *Version 2*, uses a ramp-up and hold methodology, where the TPS is pushed to its limit given a number of active threads. The test starts at a base of 50 threads, and adds 50 threads after every 100seconds of hold. Version 2 have shown to successfully triggered HPA:







High Rate Test Cases

Test Case ID	Throughput (bets/sec)	Connection Amount
HR-A-001	1000	500
HR-A-002	2500	1500
HR-A-003	3500	2500
HR-A-004	5000	4000
HR-A-005	5000	3500
HR-A-006	10000	7000
ST-001	TBD	600

Stress Testing Test Case ST-001

In the event where all test cases did not result in the discovery of the application pod's break point, to conduct further tests to gauge the server's limits.

1. Start the testing at a target load of 10,000 TPS as per HR-A-004.
2. Maintain this load for a defined observation period (e.g., 15-30 minutes) to check for stability and measure key indicators of transaction volume and response times.
3. If the system is stable, increase the TPS by 2,500.
4. Repeat step 2 and 3 until:
    - The system crashes (becomes unresponsive, critical errors occur).
    - Key indicators degrade beyond predefined acceptable thresholds.
    - Resource utilization reaches critical levels (e.g., CPU at 100% for an extended period, out-of-memory errors).
    - Record the time to crash in terms of the load (TPS) and duration of step 2 . This refers to the Betting System's ability to sustain a certain load before failing.
    - Identify the component(s) which failed.

Test Cases

More detailed test cases will be developed based on the above categories, including positive and negative scenarios to cover various aspects of the betting function.

Test Case ID	Description	Result (ms)		
BR-V2-001	Ramp and hold with increasing target TPS Target: 600 tps/threads	Median: 90 Percentile: 95 Percentile: 99 Percentile: Min: Max: TPS: ___ Total Bets: Total Time:	5017.50 7395.00 7459.00 8137.92 1368 87878 105.65 150,879 Bets 24 minute(s)	(• New) (• New) (• New) (• New) (• New) (• New)
BR-001	100 bets burst, 50 threads	Median: 90 Percentile: 95 Percentile: 99 Percentile: Min: Max: TPS: ___ Total Bets:	1178.50 1274.00 1284.00 1291.94 697 1292 77.34 100 Bets	(-46.40%) (-66.82%) (-66.72%) (-67.13%) (29.55%) (-67.34%)
BR-002	500 bets burst, 250 threads	Median: 90 Percentile: 95 Percentile: 99 Percentile:	4275.50 5049.80 5144.00 5196.00	(-4.16%) (-94.20%) (-94.12%) (-94.09%)

		Min: Max: TPS: ___ Total Bets:	811 5209 96.22 500 Bets	(-40.72%) (-94.07%)
HR-A-001	1000 bets burst, 500 threads	Median: 90 Percentile: 95 Percentile: 99 Percentile: Min: Max: TPS: ___ Total Bets: **Failed transactions:** **(Previously 0.02%)**	7382.50 11136.80 11406.80 11548.99 857 11601 86.18 1000 Bets **0**	(-27.09%) (-48.24%) (-60.16%) (-62.91%) (596.75%) (-63.21%)
HR-A-002	2500 bets burst, 1500 threads	Median: 90 Percentile: 95 Percentile: 99 Percentile: Min: Max: TPS: ___ Total Bets: **Failed transactions:** **(Previously 0.26%)**	13267.50 21181.10 21930.95 22459.00 1227 23449 98.35 2500 Bets **0**	(-24.67%) (-46.14%) (-53.46%) (-63.12%) (976.32%) (-62.51%)
HR-A-003	3500 bets burst, 2500 threads	Median: 90 Percentile: 95 Percentile: 99 Percentile: Min: Max: TPS: ___ Total Bets: **Failed transactions:** **(Previously 23.05%)**	17819.00 28502.80 30082.00 31079.98 797 31942 102.38 3500 Bets **0**	(9.64%) (-33.98%) (-52.33%) (-60.20%) (270.70%) (-60.58%)
HR-A-004	5000 bets burst, 4000 threads	Median: 90 Percentile: 95 Percentile: 99 Percentile: Min:	24534.50 39875.80 52165.95 57629.99 2741	(-48.96%) (-54.85%) (-41.73%) (-36.27%) (878.93%)

		Max: TPS: Total Bets: **Failed transactions:** **(Previously 16.42%)**	62359 80.09 4316 Bets **13.68%**	(-45.78%)
HR-A-005	5000 bets burst, 3500 threads	Median: 90 Percentile: 95 Percentile: 99 Percentile: Min: Max: TPS: Total Bets: **Failed transactions:** **(Previously 18.35%)**	27623.00 44022.50 89528.90 52966.00 2812 69529 71.70 4731 Bets **5.38%**	(-7.99%) (-37.88%) (26.16%) (-39.96%) (957.14%) (-33.78%)
HR-A-006	10000 bets burst, 7000 threads	Median: 90 Percentile: 95 Percentile: 99 Percentile: Min: Max: TPS: Total Bets: **Failed transactions:** **(Previously 23.05%)**	46756.00 72434.00 74710.45 79250.50 3109 112673 86.75 7050 Bets **29.50%**	(46.12%) (-22.24%) (-20.22%) (-20.54%) (761.22%) (-23.20%)
ST-001	Server Crashed during load test, not required to perform	Identification of failed component(s): 1. Multiple validation checks exists within /placeBet API as it is vital to prioritize reliability over raw performance. 2. Application pod requires heavy compute to complete transaction		

Conclusion

Across the board, the enhancement implemented as mentioned in the summary improved response time performance by a rough 20 to 60%, and increased the upper limits of our application from around 2000 bets/sec to more than 3500 bets/sec.

In addition, the team has made use of multiple runs of BR-V2-001 with different HPA Configurations to ensure that the application pods in production can scale horizontally, with an adjustable maximum pod configuration (current max 4). This limit can and will be increased according to the incoming traffic to support business requirements.